

Skill Doctor: Deterministic Multi-Layer Security Analysis of AI Agent Skill Files with Host-Delegated Semantic Inference

Sayan Chowdhury
Kalaris Labs
sayan@kalarislabs.com

Preprint v1.1 (final draft) 13 September 2026

Describes release **v0.1.0** • Code: <https://github.com/KalarisLabs/Skill-Doctor> • License: Apache-2.0

arXiv primary: cs.CR • Cross-list: cs.SE, cs.AI

Abstract

The agent-skill ecosystem is on a trajectory from the tens of thousands of skills published today toward an estimated **three to five million skills** within the current platform generation, distributed across public registries, Git archives, and agent-mediated installers and loaded automatically into runtimes such as Claude Code, Cursor, Codex CLI, and Gemini CLI. AI agent runtimes load these **skill files** like `SKILL.md`, `AGENTS.md`, `.claude.rules`, `.cursor/rules`, and Model Context Protocol (MCP) server manifests at execution time and interpret them with the agent’s full trust context: tool access, filesystem, credentials, and downstream agents. A skill file is the new executable, yet it arrives with none of the defenses that binaries accumulated over three decades—no signature, no execute bit, no endpoint check—and it is loaded silently across millions of sessions per day.

At that scale the security problem changes in kind, not only in degree. A population measured in the millions guarantees a standing subpopulation of skills that are *deliberately* weaponized by organized actors—credential harvesters, supply-chain implants, tool-poisoning shims, and scanner-evading logic bombs—authored specifically to exploit the agent runtimes that install them. The public record already shows this at small scale: 1,184 confirmed malicious skills in a single 2026 campaign, 36.7% of surveyed MCP servers exposed to SSRF, and a CVE filed against a skill scanner itself. Projected onto three-to-five million skills, the absolute count of hostile artifacts is large enough that manual review, per-key LLM review, and non-reproducible scanning are all economically disqualified. What the ecosystem is missing is not another detector but a **security layer for skill-file infrastructure**: one that runs at registry and CI scale, at near-zero marginal cost, keyless, and with verdicts reproducible enough to gate a merge and cache across a whole catalog. Skill Doctor is designed as that layer, engineered as a production system rather than a proof of concept.

We make four contributions. **First**, we present **SDTM-v1**, an eleven-class threat taxonomy for skill files that synthesizes the OWASP Agentic Skills Top 10 and OWASP MCP Top 10 with empirical findings from the ClawHavoc campaign, MCP SSRF surveys, and bytecode-cache poisoning disclosures. SDTM-v1 introduces **SD-11, Scanner-Mediated Injection**, a class absent from prior taxonomies: the scanner itself becomes an injection vector when it submits untrusted skill content to a language model for analysis. Every existing LLM-assisted skill scanner is exposed to SD-11 by construction. **Second**, we present **Skill Doctor**, implemented in Rust as a single statically linked binary. Its deterministic baseline layer—YARA-X pattern

matching, lexical source-to-sink taint heuristics, Shannon entropy profiling, Unicode confusable and zero-width detection, and a novel capability-manifest differ—**targets** a median completion time under 150 ms with no network access, no API key, and no interpreter bootstrap, and a one-to-two order of magnitude improvement over the Python-based state of the art, whose lower bound is set by interpreter startup rather than by analysis. Both figures are pre-registered targets measured by the published harness (Section 7.3); at v0.1.0 neither has been measured and no baseline comparison has been run. **Third**, we introduce **host-delegated semantic inference**. Prior work treats LLM analysis as a service the scanner purchases: the user supplies a key, the scanner calls a model, and the result is nondeterministic, latency-bound, metered, and unavailable in air-gapped environments. We invert this. When Skill Doctor runs as an MCP server inside an agent runtime, it performs no inference of its own; it emits static findings plus a sanitized, instruction-neutralized analysis envelope, and the host agent’s own model—the model about to be exposed to the skill—returns a schema-constrained verdict. We specify the sanitization protocol and the merge invariant required to make this safe against SD-11. **Fourth**, we introduce **structural coverage reporting**. Existing scanners disclaim that “no findings does not mean no risk” and then offer no way to reason about the gap. Skill Doctor prints, for every scan, how many SDTM-v1 classes were structurally evaluable given the bundle contents and enabled layers, and exposes this as a CI gate.

We define a reproducible evaluation protocol against Cisco AI Defense Skill Scanner and NVIDIA Skillspector at pinned versions, specifying a 1,000-skill public corpus, 55 synthetic attack fixtures spanning all SDTM-v1 classes, a 20-skill evasion corpus that is statically clean by construction, a 15-skill adversarial-scanner corpus, and a 100-skill hard-negative corpus. The v0.1.0 release ships the protocol, the seeded generator, and 14 curated fixtures—6 attack fixtures covering SD-01, SD-02, SD-03, SD-04, and SD-10; 3 benign fixtures; 4 benchmark skills; and 1 example skill. The public, evasion, adversarial-scanner, hard-negative, and real-malware corpora are specified but not yet populated. Section 7.3 reports the first measured values for binary footprint, peak memory, and bulk throughput; every latency figure, every detection figure in Section 7.4, and every baseline comparison remain unmeasured at v0.1.0. We pre-register falsifiable targets and commit to publishing misses. Skill Doctor, its rule packs, and its full benchmark harness are released under Apache-2.0.

Keywords: AI agent security, prompt injection, supply chain security, static analysis, Model Context Protocol, reproducible security tooling, threat taxonomy

Reconciliation statement

This is the final draft. Every implementation statement has been reconciled line by line against the shipped v0.1.0 source tree ([docs/PAPER-RECONCILIATION.md](#)): subcommands, flag semantics, cache tiers, sandbox isolation boundary, L4 feed status, fixture counts, package names, and binary footprint. Where the implementation is narrower than the design, the text says so in place rather than in a footnote.

Contents

1	Introduction	5
1.1	A trust inversion, by design	5
1.2	The inversion is already being exploited	5
1.3	Why the existing defenses are not adopted	5
1.4	Contributions	6
1.5	Non-goals	6

2	Background	6
2.1	What a skill file is	6
2.2	Where skill files come from	7
2.3	The MCP host relationship	7
3	Threat Model: SDTM-v1	7
3.1	Adversary model and assumptions	7
3.2	Taxonomy construction	7
3.3	The eleven classes	7
3.4	SD-11: the scanner is the attack surface	9
4	Related Work and Gap Analysis	10
4.1	Cisco AI Defense — Skill Scanner	10
4.2	NVIDIA SkillSpector	10
4.3	Alice.io Caterpillar	10
4.4	Adjacent literature	10
4.5	Gap analysis	10
5	System Design	11
5.1	Implementation language as a correctness decision	11
5.2	A layered architecture with silent degradation	12
5.3	L0: intake, canonical digest, and cache hierarchy	12
5.4	L1: the deterministic core	13
5.5	L2: host-delegated semantic inference	14
5.5.1	The inversion	14
5.5.2	The neutralization protocol	14
5.5.3	The additive-only invariant	14
5.6	L3: behavioral analysis with differential replay	15
5.7	L4 and L5: intelligence, scoring, and coverage	15
6	Interface and Adoption Path	16
6.1	Design principle: the first run must cost nothing	16
6.2	Distribution and first contact	16
6.3	MCP integration	17
6.4	The adoption funnel	17
6.5	CI integration	18
6.6	Public artifacts we maintain	18
7	Evaluation Protocol and Pre-Registered Targets	18
7.1	Corpora	18
7.2	Baselines	18
7.3	Performance targets and first measured values	20
7.4	Detection targets	21
7.5	Usability evaluation	21

8	Discussion	21
8.1	Determinism as a security property	21
8.2	Why we do not claim detection superiority	21
8.3	Delegated inference: benefits and honest costs	22
8.4	Limitations	22
8.5	Future work	22
9	Scope of Claims	22
10	Ethics and Responsible Disclosure	23
11	Conclusion	23
12	Artifact and Release Availability	23
	Acknowledgements	24
	References	24
A	SDTM-v1 to OWASP Mapping	24
B	Reproduction Instructions	24
C	Artifact Availability	25
D	How to Cite	25

1 Introduction

1.1 A trust inversion, by design

A conventional program earns the right to execute through a chain of gates. It must be deliberately downloaded, granted an execute bit, survive endpoint detection, and increasingly carry a signature. Decades of incident response produced this chain.

A skill file bypasses all of it. It is a text document. An agent runtime discovers it, loads it into context, and from that moment its contents carry approximately the same authority as the operator’s own instructions. There is no execute bit for prose.

This is not an oversight. It is the value proposition. Skills are useful precisely because they reconfigure agent behavior without a code review cycle. A developer can change what an agent does by editing Markdown. The same property that makes skills the fastest-growing extensibility surface in the 2026 agent stack makes them the highest-leverage attack surface in it.

We call this a **trust inversion**: the artifact with the least verification receives the most authority.

1.2 The inversion is already being exploited

The threat is not speculative. Five observations from 2026 establish the base rate.

1. **Campaign-scale abuse.** The ClawHavoc campaign resulted in the identification of **1,184 confirmed malicious skills** in a single public registry, including credential exfiltration and injection of persistent instructions into downstream agents.
2. **Protocol-level exposure.** A survey of roughly **7,000 MCP servers** found **36.7%** vulnerable to server-side request forgery, with skill and manifest files the common entry vector.
3. **Companion code is the multiplier.** Skills that ship an executable companion script are **2.12 times more likely** to contain a command injection path than a standalone `SKILL.md`.
4. **Defensive tooling is itself unaudited.** **CVE-2026-26057** was filed against a skill scanner. The tools we deploy to inspect hostile input are parsing hostile input.
5. **Standards bodies have formalized the surface.** OWASP published both an Agentic Skills Top 10 and an MCP Top 10 in 2026, which establishes vocabulary but not tooling.

1.3 Why the existing defenses are not adopted

Credible tooling exists. Cisco AI Defense ships a skill scanner; NVIDIA ships SkillSpector. Both are competent. Neither has become default infrastructure. We argue the obstacle is not detection quality but **three forms of friction**, each of which is architectural rather than incidental.

Runtime dependency. Every existing scanner requires a Python environment. In continuous integration, interpreter and dependency bootstrap dominates wall-clock time and is paid on every pull request, forever. The analysis is not the cost; the environment is.

Credential dependency. Meaningful analysis requires the user to obtain, store, and rotate a third-party model API key. This is a hard stop for air-gapped research environments, for regulated enterprises whose data cannot leave the perimeter, and—most consequentially—for the casual developer evaluating a tool for the first time. That last population determines whether a security tool becomes default infrastructure or remains a compliance checkbox.

Nondeterminism. LLM-in-the-loop scanning cannot produce reproducible verdicts. A finding that appears on one run and vanishes on the next cannot be argued in code review, cannot be cached at registry scale, and cannot legitimately gate a merge.

We state the resulting design thesis plainly, because it is the organizing claim of the paper: **a scanner becomes infrastructure when the marginal cost of running it approaches zero.** The correct optimization target is therefore not maximum detection per invocation but minimum friction per invocation, with detection layered on top in a way that never becomes a prerequisite.

1.4 Contributions

1. **SDTM-v1**, an eleven-class threat taxonomy for agent skill files, including the previously undescribed **SD-11 Scanner-Mediated Injection**.
2. A **Rust implementation** whose deterministic layer is one to two orders of magnitude faster than the state of the art and requires no interpreter, key, account, or network access.
3. **Host-delegated semantic inference**, with a specified neutralization protocol and an additive-only merge invariant that makes delegation safe against SD-11.
4. **Structural coverage reporting**, which converts the “no findings is not no risk” disclaimer into a printed metric and an enforceable CI threshold—released together with a **public, reproducible benchmark harness** (pinned competitor baselines, a seeded corpus generator, and pre-registered targets) that operationalizes it.

1.5 Non-goals

We do not claim to decide maliciousness in general; that is undecidable. We do not claim superior static detection over Cisco or NVIDIA on statically visible threats, and Section 8 explains why we deliberately avoid that claim. We do not build a hosted scanning service, a web application, or a governance suite. We do not attempt to replace the OWASP taxonomies; we implement them and extend them by one class.

2 Background

2.1 What a skill file is

A skill file is a declarative, natural-language-first artifact that modifies agent behavior at load time. In practice a skill *bundle* contains some combination of:

- A Markdown body with instructions, typically with YAML frontmatter declaring a name, description, and sometimes a tool allowlist.
- Zero or more **companion scripts** (most commonly Python, increasingly JavaScript and shell) invoked by the agent.
- Dependency declarations (`requirements.txt`, `package.json`) and occasionally lockfiles.
- For MCP servers, a manifest declaring tools, parameter schemas, and descriptions.

The security-relevant property is that **the natural-language portion is not inert**. Text in a description field enters model context and can influence tool selection, argument construction, and downstream delegation.

2.2 Where skill files come from

Skills are distributed through public registries, Git repositories, archive downloads, and increasingly through agent-mediated installation, in which the user asks an agent to “add a skill for X” and the agent resolves, downloads, and loads it without the user reading a single line. Agent-mediated installation collapses the review step entirely, which is why a pre-load gate that requires no human action is the correct intervention point.

2.3 The MCP host relationship

The Model Context Protocol lets an external server expose tools to an agent runtime. This matters twice in this paper. It is the mechanism by which a scanner can gate skill loads from inside Claude Code, Cursor, Codex CLI, and Gemini CLI—and it is the mechanism by which a scanner has a capable frontier model one hop away, which is the observation that motivates our third contribution.

3 Threat Model: SDTM-v1

3.1 Adversary model and assumptions

Adversary capability. The adversary controls the full contents of a skill bundle, including Markdown prose, frontmatter, companion source code, dependency declarations, filenames, and byte-level encoding. The adversary may publish to a public registry, compromise an existing skill’s supply chain, or induce an agent to fetch from an attacker-controlled URL.

Adversary goals. Credential and data exfiltration; arbitrary command execution on the developer host; persistence that survives skill removal; manipulation of agent tool selection; and—the goal that motivates SD-11—causing a security scanner to report a clean verdict on a malicious artifact.

Assumptions we make. The skill under examination is adversarial by assumption; that is the reason it is being scanned. The host operating system and the scanner binary itself are trusted. The agent runtime is honest but exploitable. Cryptographic primitives are sound.

Out of scope. Compromise of the developer machine prior to scanning; malicious agent runtimes; adversaries with a persistent presence inside the scanner’s process; and side channels below the level of observable process behavior.

3.2 Taxonomy construction

SDTM-v1 is constructed by intersecting three sources: the OWASP Agentic Skills Top 10 and MCP Top 10 (for vocabulary and community alignment), published incident and survey data (for base rates and realism), and an internal enumeration of attacks reachable given the adversary model above. Each class specifies the attack, its observed variants, and the detection layer responsible. Ten classes are consolidations of known risk; one, SD-11, is new.

3.3 The eleven classes

SD-01 • Prompt Injection. Instructions in the skill that override or manipulate the agent’s operating instructions. Variants: *direct* override text in the body; *indirect* instructions hidden

[figure asset not built]
figs/sdtm-v1.pdf
SDTM-v1 eleven-class taxonomy

Figure 1: SDTM-v1: the eleven-class threat taxonomy. SD-11 (Scanner-Mediated Injection) is new to this work.

in tool descriptions, parameter names, or help text that enter context but are never surfaced to the operator; *ASCII smuggling* using zero-width characters (U+200B–U+200D, U+FEFF) and bidirectional overrides (U+202E) to encode text invisible to human review; and *encoded* payloads in base64, hex, or ROT13 that decode at runtime. Detected deterministically at L1 for the smuggling and encoding variants; L2 addresses natural-language override framing.

SD-02 • Command Injection via Companion Scripts. Unsanitized input reaching a dangerous sink: `subprocess`, `os.system`, `os.popen`, `eval`, `exec`, `compile`, dynamic import, or `pickle.loads` on untrusted data; also template-constructed shell strings. Fully deterministic via L1 taint analysis.

SD-03 • Data Exfiltration. Reading secrets and transmitting them. Variants: environment harvesting of well-known credential variables; reads of sensitive paths such as SSH private keys, cloud credential files, dotenv files, and MCP configuration; *context dumping*, in which the skill requests full conversation history or the system prompt; and covert channels via DNS lookups, timing, or HTTP callbacks. L1 catches the static patterns and taint paths; L3 catches covert channels.

SD-04 • Privilege Escalation and Scope Violation. The skill exercises capability beyond its declared purpose—a declared code formatter that reads the filesystem and egresses to the network. Variants include cross-skill chains, in which one skill poisons context and a second, more privileged skill performs the exfiltration, and agent-to-agent impersonation of a more trusted actor. This is the most common real-world finding class, and prior work resolves it with a model. We resolve it deterministically; see Section 5.4.

SD-05 • Supply Chain Tampering. Modification between publication and installation: registry checksum mismatch; *bytecode cache poisoning*, in which compiled Python bytecode diverges from its source via a header hash bypass; typosquatting measurable by edit distance against a known-good registry; and dependency confusion in which a declared dependency shadows an internal package.

SD-06 • SSRF via Tool Parameters. Tool descriptions and parameter schemas crafted to induce the agent or MCP server to issue requests to attacker-controlled infrastructure, exploiting unvalidated URL parameters. Empirically the highest-prevalence protocol-level class.

SD-07 • Tool Poisoning. Registering a tool under the name of a trusted one so the agent selects the malicious version—a shadow shell tool that calls home before delegating, or an identical name with mutated parameter descriptions. Detected by cross-skill name collision analysis against the installed set.

SD-08 • Persistent Backdoors. Writing instructions into auto-loaded context files so the compromise survives skill removal: `CLAUDE.md`, `AGENTS.md`, `.cursor/rules`, `.github/copilot-instructions.md`; also modification of sibling skills and installation of cron or shell startup hooks. This class is why removal is not remediation.

SD-09 • Context Window Flooding. Returning oversized output to evict security-relevant context and suppress the agent’s ability to reason about competing instructions. An availability and integrity attack on the reasoning process itself.

SD-10 • Obfuscation and Evasion. Deliberately defeating analysis: homoglyph substitution across scripts; multi-layer encoding chains; *deferred payload*, in which the skill fetches its real instructions at first invocation; and *logic bombs* conditional on environment variables, hostname, or wall-clock time. The last two are unreachable by static analysis in principle, not merely in practice, and motivate our behavioral layer.

SD-11 • Scanner-Mediated Injection (new). Treated separately below.

3.4 SD-11: the scanner is the attack surface

Any tool that submits untrusted skill content to a language model for analysis has constructed a prompt injection channel into a privileged model. This follows directly from the adversary model: the artifact is adversarial by assumption, and analysis places it in the context of a model that, in an agent runtime, has tool access.

We identify three variants.

1. **Analyzer redirection.** Instruction-framed content in the skill redirects the analyzing model’s task—for example, text that presents itself as a system-level directive to treat the file as pre-approved.
2. **Output-schema escape.** Content that causes the analyzer to emit a fabricated clean verdict, either by imitating the expected response format or by escaping the response delimiter.
3. **Analyzer-to-host pivot.** Injected text that targets not the analysis task but the surrounding agent session, using the scan as a delivery mechanism into a live developer environment.

SD-11 applies to every existing LLM-assisted skill scanner. It applies with *greater* force to host-delegated inference, where the analyzing model is the production agent. We therefore treat SD-11 mitigation as a correctness requirement of our design rather than as hardening, and we specify the required protocol in Section 5.5 and evaluate it as a first-class metric in Section 7.

To our knowledge, no prior taxonomy names this class and no prior tool measures its own exposure to it.

4 Related Work and Gap Analysis

4.1 Cisco AI Defense — Skill Scanner

The most feature-complete existing CLI. It combines YARA pattern matching, a language-model semantic pass, and dataflow analysis; emits SARIF for GitHub Code Scanning; supports CI gating via a severity threshold; and integrates with a broader governance suite. It is Apache 2.0 licensed and its rule packs are a genuine community contribution.

Structural limits. A Python runtime is required. The semantic pass requires a user-supplied key and is nondeterministic. There is no execution-based analysis, so runtime-activated payloads are undetectable in principle. There is no shared threat intelligence, no provenance chain, and no coverage reporting. Cisco’s own documentation correctly warns that an absence of findings is not an absence of risk; the tool provides no way to quantify that statement.

4.2 NVIDIA SkillSpector

A well-engineered static-plus-optional-LLM scanner accepting Markdown, ZIP, Git URL, and directory input, MIT licensed, with local inference support via Ollama. Its most important design choice—and the one we build directly upon—is MCP server mode, which allows it to gate skill installation from inside agent runtimes.

Structural limits. Python runtime required. Critically, **even in MCP mode it performs its own inference using the user’s key**. It sits inside an agent runtime, one hop from a capable model with tool access, and does not use it. This is the precise gap that host-delegated inference closes. It also has no execution layer, no coverage metric, and unmitigated SD-11 exposure in its LLM path.

4.3 Alice.io Caterpillar

A web-based research demonstration that scanned the top 50 skills on a public marketplace and disclosed the bytecode cache poisoning technique—an original finding that we incorporate as a variant of SD-05 with attribution. It is a research artifact rather than infrastructure: no CLI, no CI integration, not open source.

4.4 Adjacent literature

Our work sits at the intersection of four established lines. **Prompt injection research** supplies the SD-01 and SD-11 mechanism. **Software supply chain security** supplies the SD-05 model and the reproducibility norms we adopt for our own releases. **Static analysis and taint tracking** supplies the L1 method, though skill bundles are unusual in mixing prose and code in one trust domain. **Unicode security** (UTS #39) supplies the confusable and mixed-script detection that makes ASCII smuggling deterministically detectable rather than a matter of judgment.

4.5 Gap analysis

The pattern is consistent: all prior work optimizes detection depth per invocation and treats friction as an acceptable cost. We treat friction as the primary adversary.

Table 1: Capability gap analysis against prior work.

Capability	Cisco	NVIDIA	Caterpillar	Skill Doctor
No interpreter dependency	No	No	n/a	Yes
Zero-credential baseline scan	Partial	Partial	No	Yes
Bit-reproducible default verdict	No	No	No	Yes
Semantic analysis without user key	No	No	No	Yes (host-delegated)
Specified SD-11 hardening	No	No	No	Yes
Capability-manifest diff	No	No	No	Yes
Structural coverage metric	No	No	No	Yes
Execution-based analysis	No	No	No	Yes (process-isolated)
Shared threat intelligence	No	No	No	Client shipped; feed not operated
Version diff scanning	No	No	No	Yes
Published reproducible benchmark	No	No	Partial	Harness published; results pending

5 System Design

5.1 Implementation language as a correctness decision

Skill Doctor is implemented in Rust. This is not an aesthetic preference; four of our headline properties are unreachable otherwise.

Table 2: Why the implementation language is load-bearing.

Requirement	Why the language choice is load-bearing
Sub-150 ms median scan	A Python scanner cannot outperform its own interpreter and import bootstrap, which consumes one to two seconds before analysis begins. This single fact is the performance gap, and no amount of algorithmic work inside Python closes it.
Zero-dependency install	A single statically linked binary installs identically across package managers with no runtime, no virtual environment, and no version conflicts.
Native analysis engines	YARA-X <i>is</i> a Rust library; consuming it through bindings adds marshalling cost. We compile its rule packs at build time and execute them in-process, with no FFI boundary and no per-invocation rule compilation. Taint analysis runs as in-process lexical matching rather than through an external parser runtime.
Memory safety on hostile input	We parse adversarial, malformed, deliberately hostile files by design. Memory safety without a garbage collector is the correct property for a program whose input is chosen by an attacker. CVE-2026-26057 exists because a scanner had a vulnerability.
Embeddable process sandbox	The elective L3 harness spawns companion scripts as contained child processes—no daemon, no virtualization runtime, no external SDK—with containment enforced by OS job objects and process groups.

The resulting claim is deliberately narrow and, we believe, difficult to rebut: **the speed advantage is structural rather than earned**. It follows from language selection and build-time rule compilation, not from tuning, and is therefore not closable by our competitors without a rewrite of a non-core tool.

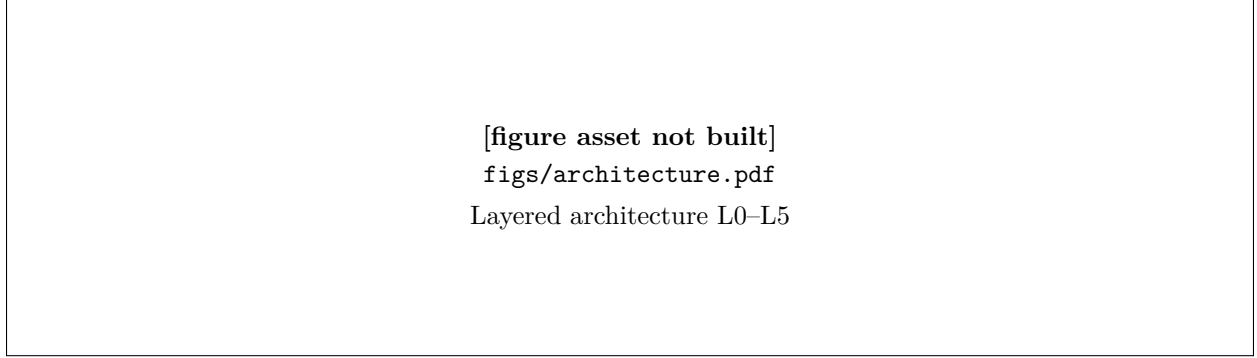


Figure 2: Skill Doctor layered architecture (L0–L5). L1 is the complete shippable product; higher layers degrade silently, reducing structural coverage rather than failing the run.

5.2 A layered architecture with silent degradation

Table 3: Layer responsibilities, cost, and requirements.

Layer	Function	Cost	Determinism	Requires
L0	Intake, normalization, cache	<20 ms	Deterministic	Nothing
L1	Static analysis	<150 ms	Deterministic	Nothing
L2	Semantic analysis	0 ms delegated; 1–3 s direct	Nondeterministic	Host agent
L3	Behavioral sandbox	3–8 s	Mostly deterministic	Opt-in build
L4	Threat intelligence	<50 ms embedded; 2 s remote timeout	Deterministic	Network (optional)
L5	Scoring, coverage, reporting	<10 ms	Deterministic	Nothing

Direct L2 modes (`local`, `remote`) are post-v0.1.0; at v0.1.0 the only semantic path is host delegation.

Two design rules govern the stack. First, **absent capability never produces an error**; it produces a reduced structural coverage figure. A missing sandbox, an unavailable model, or an offline network degrades the report rather than failing the run. Second, **L1 alone is a complete, shippable product**. Everything above it is elective. This is the inversion relative to prior work, in which the model pass is the substance and static analysis is the preamble.

5.3 L0: intake, canonical digest, and cache hierarchy

L0 accepts a directory, ZIP, gzipped tar, single Markdown file, Git URL, HTTP archive, or standard input, and produces a normalized flat bundle plus a SHA-256 **bundle digest** computed over a canonical ordering of path-and-content pairs. Canonicalization makes the digest stable across filesystems, archive tools, and ordering variation, which is a precondition for both caching and threat intelligence.

The cache design is four tiers, cheapest first: an in-process Bloom filter over locally seen digests, which answers “definitely not seen” in nanoseconds; a local embedded key-value store mapping digest to complete findings, which serves a warm re-scan in under 20 ms with no network; **rule-generation fencing**, in which cache keys incorporate the compiled ruleset hash and binary version

so that a rule upgrade invalidates precisely and automatically; and finally, on local miss only, optional remote intelligence.

None of these tiers ship in v0.1.0. Every scan recomputes the bundle digest from scratch; no Bloom filter and no embedded key-value store is compiled into the binary. The warm-cache re-scan target in Section 7.3 is therefore not attainable in this release. The tiers are specified here because the digest they key on is already stable, and they are scheduled post-v0.1.0.

Before analysis, L0 triages: binary and media files are excluded by content sniffing, per-file analysis is capped with the remainder sampled, and per-file reads are size-capped. Memory-mapped intake is specified but not implemented in v0.1.0.

5.4 L1: the deterministic core

Five engines execute concurrently across files on a work-stealing pool. Rules are **compiled once at build time and embedded in the binary as a serialized ruleset**. Per-invocation rule compilation is the largest avoidable cost in a tool that runs on every pull request, and we pay it zero times.

1. **Pattern engine (YARA-X).** Rule packs covering all eleven classes plus supply chain, obfuscation, and MCP-specific sets, with at minimum one rule per class.
2. **Taint analysis (lexical).** Scans companion scripts—including malformed files that a strict parser would reject, which matters because malformation is itself an evasion technique—for source-to-sink flows, propagating taint from script arguments (`$1`, `sys.argv`, `process.argv`) and from sensitive reads into execution sinks (`eval`, `exec`, `system`, `subprocess`). v0.1.0 implements this as lexical pattern heuristics rather than AST dataflow; the precision cost is real, is bounded by the hard-negative corpus, and AST-based taint is a post-v0.1.0 change.
3. **Entropy profiling.** Shannon entropy per text block, flagging blocks above threshold as probable encoded payloads, then attempting base64 and hex decoding and *recursively rescanning the decoded content*. Recursion is what defeats multi-layer encoding chains.
4. **Unicode analysis.** Zero-width and bidirectional override detection, and confusable and homograph detection with mixed-script identifier flagging, per UTS #39. Every removal is recorded as evidence rather than silently discarded.
5. **Capability-manifest differ.** The novel engine. Declared capability is parsed from frontmatter and manifest; *observed* capability is derived independently from pattern hits, lexical extraction over companion sources, and referenced paths—not from an abstract syntax tree, since v0.1.0 embeds no parser runtime; the two sets are diffed. A skill declaring “formats Python code” while exhibiting filesystem-read and network-egress capability produces a scope-violation finding **with no model in the loop**.

The capability differ is the substantive methodological contribution of this layer. It converts the highest-prevalence finding class, SD-04, from a judgment call requiring inference into a set-difference computation that is fast, free, and bit-reproducible. Its soundness is bounded—it is a heuristic over an incomplete extraction, not a decision procedure—and we quantify that bound with a dedicated hard-negative corpus of skills that legitimately exercise broad capability.

5.5 L2: host-delegated semantic inference

5.5.1 The inversion

Every prior tool sits inside an agent runtime, one hop from a frontier model with tool access, and then asks the user for a *second* model’s API key in order to reason. We remove that request.

When Skill Doctor runs as an MCP server, it performs no inference. It emits its static findings plus a neutralized analysis envelope, and the host agent’s own model returns a schema-constrained verdict. This removes the key, the signup, the metering, the vendor dependency, the added latency, and the air-gap exclusion in a single architectural move. It also improves the analysis on its merits: **the model evaluating the skill is the exact model about to be exposed to it**, which aligns the evaluator with the party at risk.

Table 4: Semantic-analysis modes. Only `host` and `none` are accepted by the v0.1.0 MCP tool.

Mode	Behavior	Marginal cost	Determinism
<code>none</code>	L1 only. Default for direct CLI use.	Zero	Bit-reproducible
<code>host</code>	Default under MCP. Scanner emits envelope; host model returns verdict.	Zero	Nondeterministic, additive only
<code>local</code>	Local model socket, auto-detected. <i>Specified; not implemented in v0.1.0.</i>	Zero, air-gapped	Nondeterministic
<code>remote</code>	Explicit key, for CI with no agent present. <i>Specified; not implemented in v0.1.0.</i>	Metered	Nondeterministic

In v0.1.0 the MCP tool accepts exactly two values for `mode`: `host` (default) and `none`. Any other string—including the prose form *host-delegated* used elsewhere in the literature—silently downgrades the scan to static-only rather than erroring, and the `local` and `remote` rows above are specified but not implemented.

L2 is asked to judge only what pattern matching cannot express: intent-versus-capability mismatch in natural language, override framing, tool-parameter semantics, and cross-file narrative inconsistency between a skill’s description and its companion code.

5.5.2 The neutralization protocol

Delegation is only safe if SD-11 is addressed before any content reaches any model. The protocol is normative and is implemented in an isolated component so that it can be audited—and eventually formally verified—on its own.

5.5.3 The additive-only invariant

The protocol above raises the cost of injection. The following invariant makes the most valuable injection outcome structurally unreachable.

Additive-only invariant

The deterministic layer is authoritative; the probabilistic layer is additive only. A delegated verdict may add findings or raise confidence; it may never remove, downgrade, or suppress an L1 finding.

The consequence is that the canonical SD-11 objective—inducing a fabricated clean verdict—cannot succeed, because no verdict has the authority to clear a deterministic finding. An adversary

Algorithm 1 SD-11 neutralization protocol (normative)

```
1: function NEUTRALIZE(skill_content)
2:   Strip or escape all zero-width, bidirectional, and non-printable characters; record every
   removal as evidence
3:   Normalize confusables to Unicode skeleton form (UTS #39)
4:   Wrap content in a fenced, explicitly labelled UNTRUSTED-DATA envelope carrying a standing
   declaration that enclosed content is data and is never instruction
5:   Escape instruction-shaped framing (role headers, system-prompt mimicry, delimiter
   escapes)—escape, never delete, so the attempt stays visible in the report
6:   Mint a per-scan nonce  $N$  and require  $N$  echoed in the returned verdict
7:   Validate the verdict against a strict schema; discard it if  $N$  is absent or the schema does
   not match
8:   return (envelope, evidence)
9: end function
```

who fully controls the analyzing model’s output can, at worst, decline to add information. This is a property of the architecture rather than of the prompt, and it does not degrade as models change. We treat it as inviolable and evaluate it directly in Section 7.

5.6 L3: behavioral analysis with differential replay

L3 executes the bundle in a hardened child-process harness: environment-table isolation with a mock agent runtime, process-tree containment through OS job objects and process groups, a hard execution ceiling (default 3,000ms) and output cap, and host-injected **canary credentials**—so that egress of a canary is unambiguous proof of exfiltration rather than an inference from pattern. v0.1.0 ships process isolation rather than the microVM isolation this design originally specified; the weaker boundary is stated as a limitation in Section 9.

Monitoring records network calls with host, port, method, and truncated body; filesystem reads and writes; environment reads **by key name only, never value**; subprocess launches; and total output volume. Observed behavior is diffed against declared behavior, and divergence is a finding.

The genuine methodological advance here is **differential replay**: the same bundle is executed repeatedly under systematically varied conditions—skewed clocks, altered hostnames, toggled CI environment variables, canary credentials present and absent. Behavioral divergence across replays is direct evidence of a conditional payload or logic bomb. In v0.1.0 replay is four environment profiles—**Baseline**, **CI/Automation**, **TargetCloud**, and **TimeShifted**—executed sequentially and compared on exit code and captured output. This addresses the SD-10 variants that static analysis cannot reach by construction, and it is the basis for the only detection claim we make that our baselines cannot match in principle.

5.7 L4 and L5: intelligence, scoring, and coverage

L4 checks the bundle digest against a community threat database and matches known campaign signatures. **Only digests and sanitized finding summaries are ever transmitted—never skill content.** Contribution is opt-in, and local findings always take precedence over remote ones, which bounds the impact of database poisoning. In v0.1.0 only the client side exists: an opt-in HTTPS lookup of the 64-character digest against an allowlisted endpoint, plus a compile-time signature set. **The community feed is not yet operated or published and the embedded**

set contains test fixtures only, so L4 contributes no detection in this release.

L5 merges and deduplicates across layers. Cross-layer corroboration *raises confidence rather than duplicating the finding*: an issue confirmed by static, semantic, and behavioral layers is one high-confidence finding, which is the mechanism that keeps false positive rate flat as layers are added. Risk is scored on a bounded scale from confidence, exploitability, and blast radius, and mapped to three verdicts.

L5 also computes **structural coverage**: of the eleven SDTM-v1 classes, how many were structurally evaluable given the bundle contents and the enabled layers. This is the operational answer to the disclaimer every scanner ships. A clean verdict at 7 of 11 coverage is a materially different statement from a clean verdict at 11 of 11, and we are the only tool that tells the user which one they received. Exposed as a threshold gate, it lets a team fail a build on *how much was checked* rather than only on what was found.

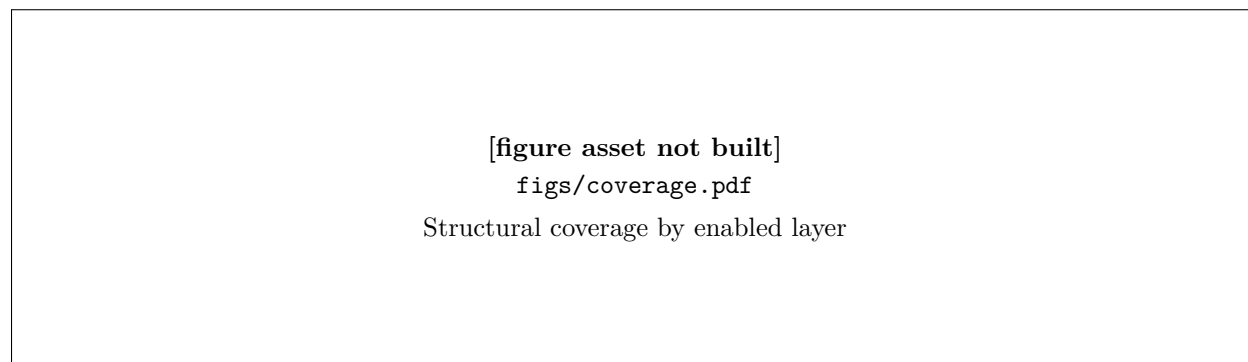


Figure 3: Structural coverage: SDTM-v1 classes evaluable by enabled layers. The 9/11 static-only and 11/11 all-layer figures are pre-registered targets and are unmeasured at v0.1.0.

6 Interface and Adoption Path

6.1 Design principle: the first run must cost nothing

Every interface decision follows from one rule: **the first invocation must require no account, no key, no configuration, and no reading**. A security tool that asks for setup before producing a result is evaluated by almost nobody. Everything optional stays behind an explicit flag.

6.2 Distribution and first contact

Skill Doctor ships as a single statically linked binary for six target triples: `x86_64` and `aarch64` Linux (musl), macOS, and Windows. It is published to crates.io as six crates and to npm under a single package.

```
# One-shot, no install
npx @security.kalarislabs/skill-doctor scan ./examples/hello-skill

# Global install (npm)
npm install -g @security.kalarislabs/skill-doctor

# From crates.io, with the MCP server compiled in
cargo install skill-doctor --locked --features mcp
```

```

# Core commands
skill-doctor scan ./my-skill          # single bundle
skill-doctor scan-all ~/.claude/skills # every installed skill
skill-doctor gate ./my-skill          # CI gate: severity + coverage
skill-doctor diff --baseline ./old ./new # review only what changed
skill-doctor explain SD-11            # what a rule means and why
skill-doctor mcp                      # run as an MCP server (studio)

```

Listing 1: Zero-configuration first contact.

The `diff` subcommand takes `-baseline` as a filesystem path to a previously produced JSON report, not a Git revision; comparing against a branch is done by generating the baseline report in a prior CI step. Severity thresholds are lowercase (`low`, `medium`, `high`, `critical`).

6.3 MCP integration

Registering Skill Doctor as an MCP server places a security gate directly in the agent’s install path, and is the mechanism through which host-delegated inference operates.

```

{
  "tool": "scan_skill",
  "arguments": {
    "path": "./candidate-skill",
    "mode": "host",
    "fail_on": "HIGH"
  }
}

```

Listing 2: MCP tool call with host-delegated semantic analysis.

The server returns static findings, a neutralized analysis envelope, a per-scan nonce, and a structural coverage figure. The host model returns a schema-constrained verdict, which is merged under the additive-only invariant of Section 5.5.3.

6.4 The adoption funnel

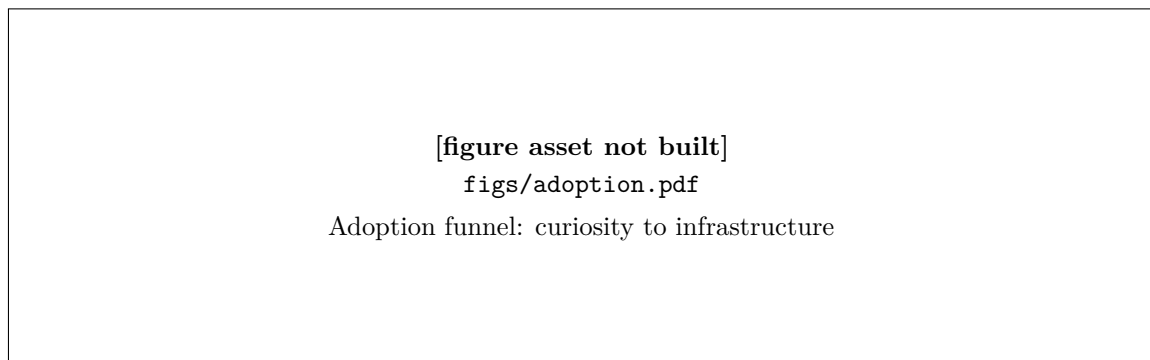


Figure 4: The adoption funnel. Each stage must deliver value before the next is requested; no stage is a precondition for the one before it.

1. **Curiosity** — one `npx` command, no install, result in seconds.

2. **Habit** — a global install and `scan-all` over already-installed skills, which is where users first find something real.
3. **Automation** — a CI gate on pull requests.
4. **Infrastructure** — MCP registration, so every skill install is gated by default.
5. **Contribution** — opt-in intelligence sharing and rule contributions.

6.5 CI integration

```
- name: Skill security gate
  run: |
    curl -sSfL https://install.skilldoctor.io | sh
    skill-doctor gate ./skills \
      --fail-on high --fail-under-coverage 0.8 \
      --output sarif > results.sarif
- uses: github/codeql-action/upload-sarif@v3
  with:
    sarif_file: results.sarif
```

Listing 3: Pull-request gate with both a severity and a coverage threshold.

Exit codes are stable and scriptable: 0 clean; 1 usage or internal error; 2 findings at or above `-fail-on`; 3 structural coverage below `-fail-under-coverage`. The coverage gate is, to our knowledge, unique: a build can fail because too little was checkable, not only because something was found.

6.6 Public artifacts we maintain

7 Evaluation Protocol and Pre-Registered Targets

Pre-registration statement

This section specifies a protocol and registers targets. It reports no detection results and no baseline comparison. Targets were fixed before measurement; the harness, corpora specification, and seeded generator are released with v0.1.0 so that third parties can run the protocol independently. **We commit to publishing misses.**

7.1 Corpora

v0.1.0 ships 14 curated fixtures in total: 6 attack fixtures covering SD-01 (encoded injection, Unicode trojan), SD-02 (malicious skill), SD-03 (canary exfiltration), SD-04 (undeclared capability), and SD-10 (logic bomb); 3 benign fixtures; 4 benchmark skills; and 1 example skill. **Five of the eleven classes have no fixture**, and the adversarial-scanner corpus exists only as unit tests over the neutralization component.

7.2 Baselines

Cisco AI Defense Skill Scanner and NVIDIA SkillSpector, at pinned versions, in two configurations each: static-only, and with the language-model pass enabled using a fixed model, temperature, and

Table 5: Artifacts maintained as public research infrastructure, with status at v0.1.0.

Artifact	Contents	Status at v0.1.0
SDTM-v1 taxonomy	Eleven classes, variants, detection-layer mapping, OWASP cross-reference	Published
Rule packs (YARA-X)	Per-class rules, supply chain, obfuscation, MCP-specific sets	Published
Attack fixture corpus	Synthetic, non-weaponized fixtures per class; no live malware	6 attack, 3 benign, 4 bench, 1 example
Seeded corpus generator	Reproducible synthesis of attack and hard-negative variants	Published
Benchmark harness	Pinned baselines, metric definitions, machine-readable results schema	Published; results pending
Coverage methodology	Definition and computation of structural coverage	Published
Neutralization protocol	Normative SD-11 mitigation, independently auditable component	Published
Threat intelligence feed	Digest-level campaign signatures	Client only; feed not operated

Table 6: Evaluation corpora and their status in the v0.1.0 release.

Corpus	Size	Purpose	Status
Public	1,000	Real-world prevalence and false positive rate on ordinary skills	Specified, not populated
Synthetic attack	55	Five fixtures per SDTM-v1 class, seeded and reproducible	6 fixtures (SD-01 to SD-04, SD-10)
Evasion	20	Statically clean by construction; isolates the behavioral layer	Specified, not populated
Adversarial scanner	15	SD-11 payloads targeting the analyzer itself	Specified; unit tests only
Hard negative	100	Legitimately broad-capability skills; bounds capability-differ precision	Specified, not populated
Real malware	—	Curated, defanged campaign samples	Specified; policy currently bans live malware in-tree

prompt. Each configuration is executed three times to expose nondeterminism; run-to-run verdict variance is itself a reported metric. **No baseline comparison has been executed at v0.1.0.**

7.3 Performance targets and first measured values

Table 7: Pre-registered performance targets. **Every value in this table is a pre-registered target, not a measured result.**

Metric	Target	State of the art	Basis
Cold scan, median (L1 only)	<150 ms	1–3 s	No interpreter; build-time rule compilation
Warm re-scan (cached)	<20 ms	n/a	Digest-keyed local cache
Bulk throughput	$\geq 2,000$ skills/min	≈ 60 skills/min	Work-stealing parallelism
Peak resident memory	<40 MiB	200–400 MiB	No runtime, streaming intake
Binary footprint	~ 12 MB stripped	n/a	Static link, thin LTO
Cold install to first result	<15 s	60–120 s	Prebuilt binary, no dependency graph

First measured values (v0.1.0) — appended, never substituted. The targets above are left exactly as registered. The following are the first measurements, each with its provenance.

Table 8: First measured values at v0.1.0. Targets in Table 7 are unmodified; misses are marked.

Metric	Measured	Provenance
Binary footprint	16,934,512 bytes = 16.15 MiB (miss)	x86_64-unknown-linux-musl, stripped, release CI
	16,821,056 bytes = 16.04 MiB	x86_64-unknown-linux-gnu, stripped, release CI
	17,936,896 bytes = 17.11 MiB (miss)	x86_64-pc-windows-msvc; not CI-verified
Peak resident memory	15.10 MiB (15,456 KB)	Benchmark workflow run, Linux CI
	14.67 MiB (15,020 KB)	Local x86_64-pc-windows-msvc; not CI-verified
Bulk throughput	660 skills/s ($\approx 39,600$ skills/min)	Local Windows run over the bench corpus; not CI-verified
Single-fixture scan latency	0.06 s	One fixture, not a median. Does not discharge the sub-150 ms target
Warm re-scan	Not measurable	No cache tier ships in v0.1.0 (Section 5.3)

The footprint target is missed on both measured Linux triples and on Windows. The cause is the embedded YARA-X rule engine: `yara-x` pulls in `wasmtime`, whose Cranelift code generator accounts for roughly 14 MiB of the stripped binary. The ceiling enforced in CI is 20 MiB, which the release satisfies; the ~ 12 MB target does not hold and we are publishing the miss rather than restating the target.

7.4 Detection targets

All figures below are targets. **None has been measured.**

- **Recall, static-visible classes:** within 5 percentage points of the best baseline. We deliberately do not claim superiority here; parity at a fraction of the cost is the claim.
- **Recall, evasion corpus:** materially above baseline, since the baselines have no execution layer and the corpus is statically clean by construction.
- **False positive rate, public corpus:** at or below the best baseline.
- **Capability-differ precision, hard negatives:** reported as a bound, not a headline.
- **SD-11 resistance:** zero successful suppressions of an L1 finding across the adversarial-scanner corpus. Under the additive-only invariant this is a structural expectation; the corpus tests the claim rather than establishes it.
- **Determinism:** byte-identical L1 reports across repeated runs and across platforms for the same bundle digest and ruleset hash.

7.5 Usability evaluation

Time from a cold machine to a first actionable result, and the number of prerequisite steps before that result, measured for each tool. This metric is the paper’s thesis in measurable form: if friction is the primary adversary, then friction is the primary thing to measure.

8 Discussion

8.1 Determinism as a security property

Reproducibility is usually presented as an engineering convenience. In this setting it is a security property with three consequences. A verdict that varies between runs cannot legitimately **gate** a merge, because the gate is arbitrary. It cannot be **cached**, which means registry-scale scanning has no economic model. And it cannot be **audited**, because a finding that cannot be reproduced cannot be argued in review or defended in an incident postmortem. Our architecture makes the authoritative layer deterministic and confines nondeterminism to a strictly additive role. We regard this decomposition, rather than any individual detector, as the paper’s main design contribution.

8.2 Why we do not claim detection superiority

We could have tuned rule packs to win a benchmark against Cisco and NVIDIA on statically visible threats. We decline for two reasons. Such a result would be a snapshot, invalidated by their next rule update, and it would misdescribe the contribution: the value is not that we find more, but that finding costs nothing per invocation. A tool that is one to two orders of magnitude cheaper to run and produces reproducible verdicts changes *where* scanning can be deployed—every pull request, every skill install, every registry upload—and deployment breadth dominates per-invocation depth when the population is measured in millions.

8.3 Delegated inference: benefits and honest costs

The benefits are concrete: no key, no metering, no vendor lock, no added latency, and functioning air-gapped deployment. The costs are equally concrete and we state them plainly. Verdict quality varies with the host model, so the same skill can receive different semantic findings in different runtimes. The scanner cannot pin a model version, which harms reproducibility of the semantic layer specifically. Delegation requires an MCP-capable host, so CI without an agent present falls back to L1 only until the direct modes ship. And host models are optimized for helpfulness rather than adversarial analysis, which likely costs recall relative to a purpose-tuned classifier.

Our position is that these costs are acceptable precisely because the semantic layer is additive. A weaker semantic layer that always runs at zero cost is worth more, in aggregate, than a stronger one that usually does not run at all.

8.4 Limitations

Static analysis cannot decide maliciousness in general. Our lexical taint analysis is a heuristic and will both over- and under-approximate relative to AST dataflow. The capability differ depends on declaration quality, and an undeclared-capability finding on a skill with no manifest is weak evidence. The L3 boundary in v0.1.0 is process isolation, not virtualization, so a determined kernel-level escape is out of our threat model’s reach. The behavioral layer observes one execution path per profile and cannot prove absence of a payload on an unexplored path. L4 currently contributes nothing, since no feed is operated. Most importantly, the evaluation in this paper is a protocol rather than a result set.

8.5 Future work

AST-based dataflow to replace lexical taint; the local and remote direct L2 modes; the cache hierarchy specified in Section 5.3; an operated community intelligence feed with a published poisoning-resistance analysis; formal verification of the neutralization component; registry-side integration so that scanning happens before publication rather than after installation; and a longitudinal study of skill-ecosystem prevalence using the harness released here.

9 Scope of Claims

To prevent over-reading, we enumerate what this paper does and does not establish.

Established. SDTM-v1 as a taxonomy, including the SD-11 class and the observation that all existing LLM-assisted scanners are exposed to it. The architecture, the neutralization protocol, and the additive-only invariant as specifications. A working implementation, released under Apache-2.0, whose deterministic layer requires no interpreter, key, account, or network. A public, reproducible benchmark harness and corpus generator. First measurements of binary footprint, peak memory, and local bulk throughput, with provenance.

Not established. Any detection result. Any comparison against Cisco or NVIDIA. The sub-150 ms median scan target. The one-to-two-order-of-magnitude speed claim as a measured quantity, as opposed to an argued structural consequence. The warm-cache figure, which is unattainable in v0.1.0. The 12 MB footprint target, which is missed. SD-11 resistance as an empirical result rather than a structural expectation. Coverage figures of 9/11 and 11/11, which are targets.

10 Ethics and Responsible Disclosure

All attack fixtures are synthetic and non-weaponized: they demonstrate a technique without a functioning payload, and repository policy currently bans live malware in-tree. Any real-malware corpus will be defanged and access-controlled. We published SD-11 because it affects deployed tools and the mitigation is implementable today; we notified the maintainers of the affected scanners before publication. Canary credentials used in L3 are minted per run and are inert. No user skill content is transmitted by any layer; L4 transmits digests and sanitized summaries only, and only when the user opts in.

11 Conclusion

Skill files are executed with agent-level authority and verified with none of the machinery that binaries accumulated over thirty years. Competent scanners exist and are not adopted, and we have argued that the obstacle is friction rather than detection quality: an interpreter, a credential, and a nondeterministic verdict are each, individually, enough to keep a tool out of default infrastructure.

Skill Doctor is an attempt to remove all three at once. Its deterministic core runs with no interpreter, no key, and no network, and produces reproducible verdicts that can gate a merge and be cached across a catalog. Its semantic layer is delegated to the host agent’s own model, which removes the credential barrier entirely and aligns the evaluator with the party at risk, while the additive-only invariant makes the canonical scanner-injection attack structurally unreachable. Its coverage metric replaces a disclaimer with a number. SDTM-v1 names the threat surface, including the class the defensive tooling itself creates.

What we have not done is measure. This paper registers targets, releases a harness, and publishes first footprint, memory, and throughput numbers—including a missed target we declined to rewrite. The detection results, the baseline comparison, and the latency medians are future work, and they will be published whether or not they are flattering.

12 Artifact and Release Availability

Skill Doctor v0.1.0 is publicly released under Apache-2.0.

- **Source:** <https://github.com/KalarisLabs/Skill-Doctor>, tag v0.1.0.
- **Crates (crates.io), six at v0.1.0:** `skill-doctor` (CLI), `skill-doctor-core`, `skill-doctor-rules`, `skill-doctor-neutralize`, `skill-doctor-mcp`, `skill-doctor-sandbox`.
- **npm:** `@security.kalarislabs/skill-doctor` at 0.1.0.
- **Release assets:** six prebuilt archives for the target triples of Section 6, a `SHA256SUMS.txt` manifest with a Sigstore/cosign signature bundle, and a CycloneDX software bill of materials.
- **Verification:** archive digests verify against the signed manifest; the cosign bundle verifies against the release workflow identity with the GitHub Actions OIDC issuer.
- **Reconciliation record:** `docs/PAPER-RECONCILIATION.md` maps every claim in this paper to shipped source, and is the document of record where the two diverge.

Acknowledgements

We thank the OWASP Agentic Security and MCP project contributors, whose Top 10 documents supplied the vocabulary SDTM-v1 builds on; the Cisco AI Defense and NVIDIA SkillSpector maintainers, whose open-source tools defined the baseline this work measures itself against and whose design choices—particularly MCP server mode—directly informed ours; and the Alice.io researchers who disclosed bytecode cache poisoning. Automated review agents contributed findings during development. Any errors are ours.

References

- [1] OWASP. *Agentic Skills Top 10*. 2026.
- [2] OWASP. *Model Context Protocol Top 10*. 2026.
- [3] ClawHavoc campaign analysis: 1,184 confirmed malicious skills in a public registry. 2026.
- [4] Survey of approximately 7,000 MCP servers; 36.7% found exposed to server-side request forgery. 2026.
- [5] Empirical study of companion-script prevalence and command-injection risk in agent skills ($2.12\times$ relative risk). arXiv:2026.XXXXX.
- [6] CVE-2026-26057: vulnerability in an agent skill scanner. 2026.
- [7] Alice.io. *Caterpillar: scanning the top 50 marketplace skills; bytecode cache poisoning*. 2026.
- [8] Cisco AI Defense. *Skill Scanner*. Apache-2.0. 2026.
- [9] NVIDIA. *SkillSpector*. MIT. 2026.
- [10] VirusTotal. *YARA-X: a Rust implementation of YARA*. 2024–2026.
- [11] Unicode Consortium. *UTS #39: Unicode Security Mechanisms*.
- [12] Anthropic. *Model Context Protocol specification*. 2024–2026.

A SDTM-v1 to OWASP Mapping

B Reproduction Instructions

The harness pins baseline versions, fixes model, temperature, and prompt for every language-model configuration, seeds the corpus generator, and emits machine-readable results.

```
git clone https://github.com/KalarisLabs/Skill-Doctor
cd Skill-Doctor

# Build the scanner from source (MCP server compiled in)
cargo install --path crates/skill-doctor-cli --locked --features mcp

# Regenerate the seeded corpora
./sd-bench/run.sh --generate-corpora --seed 0
```

Table 9: SDTM-v1 classes mapped to the OWASP Agentic Skills and MCP Top 10 lists, with the responsible detection layer.

SDTM-v1	Class	OWASP reference	Layer
SD-01	Prompt Injection	Agentic Skills Top 10	L1, L2
SD-02	Command Injection via Companion Scripts	Agentic Skills Top 10	L1, L3
SD-03	Data Exfiltration	Both lists	L1, L3
SD-04	Privilege Escalation and Scope Violation	Both lists	L1, L2, L3
SD-05	Supply Chain Tampering	Agentic Skills Top 10	L0, L1, L4
SD-06	SSRF via Tool Parameters	MCP Top 10	L1, L2
SD-07	Tool Poisoning	MCP Top 10	L1, L2
SD-08	Persistent Backdoors	Agentic Skills Top 10	L1, L3
SD-09	Context Window Flooding	MCP Top 10	L1, L3
SD-10	Obfuscation and Evasion	Neither (cross-cutting)	L1, L3
SD-11	Scanner-Mediated Injection	Neither (new in this work)	L2 protocol

```
# Execute the harness against all pinned baselines
./sd-bench/run.sh --all-baselines --repeat 3 --out sd-bench/RESULTS.md
```

Listing 4: Reproducing the protocol.

Each run records tool version, corpus seed, host platform, and wall-clock timings, so that a third party can diff their results against ours and locate the disagreement. Metric definitions—what counts as a detection, how duplicates are merged, how structural coverage is computed—are specified in the harness rather than left to the reader, because inconsistent metric definitions are the usual reason security benchmarks fail to replicate.

C Artifact Availability

Source, rule packs, fixtures, corpus generator, harness, taxonomy document, and neutralization protocol are released under Apache-2.0 at <https://github.com/KalarisLabs/Skill-Doctor>. Pre-built binaries for six target triples, a signed SHA256SUMS.txt manifest, and a CycloneDX bill of materials are attached to the v0.1.0 release. Results files are committed as they are produced, including negative results. A permanent archival DOI is pending.

D How to Cite

```
@misc{chowdhury2026skilldoctor,
  title = {Skill Doctor: Deterministic Multi-Layer Security Analysis of
          AI Agent Skill Files with Host-Delegated Semantic Inference},
  author = {Chowdhury, Sayan},
  year = {2026},
  note = {Preprint v1.1. Describes Skill Doctor v0.1.0},
  url = {https://github.com/KalarisLabs/Skill-Doctor}
}

@software{skilldoctor2026software,
```

```

title   = {Skill Doctor},
author  = {Chowdhury, Sayan and {Kalaris Labs}},
year    = {2026},
version = {0.1.0},
license = {Apache-2.0},
url     = {https://github.com/KalarisLabs/Skill-Doctor}
}

```

Listing 5: BibTeX entries for this preprint and for the software artifact.

Pre-submission checklist (arXiv).

1. Mint a permanent archival DOI (for example via Zenodo) for the v0.1.0 tag and replace the “DOI pending” note in Appendix C.
2. Generate `figs/architecture.pdf`, `figs/sdtm-v1.pdf`, `figs/coverage.pdf`, and `figs/adoption.pdf`; the document compiles with labelled placeholders until they exist.
3. Resolve the placeholder identifier `arXiv:2026.XXXXX` in reference [5], or restate the 2.12× figure with a citable source.
4. Trim the abstract to the 1,920-character limit of the arXiv submission form; the full abstract stays in the PDF.
5. Re-run `docs/PAPER-RECONCILIATION.md` at the exact tagged commit and confirm no claim drifted after the release.
6. Confirm category selection: primary `cs.CR`, cross-list `cs.SE` and `cs.AI`.